

Tarea

Desarrollo Web en Entorno Servidor

Jugadores de Futbol

Unidad 5: Programación orientada a objetos en PHP.

Abraham Quintana Micó 3º DAW Semi B

Índice

Tarea Desarrollo Web en Entorno Servidor.....	1
Introducción.....	3
Estructura del proyecto.....	4
Dependencias instaladas y justificación de cambios.....	6
Explicación de los ficheros desarrollados.....	7
Explicación de cada pantalla.....	9
Mejoras añadidas y justificación.....	13
Conclusión.....	14
Bibliografía.....	15

Introducción

En esta práctica he desarrollado una aplicación web en PHP orientada a objetos para la gestión de jugadores de un equipo de fútbol.

El objetivo principal era poner en práctica la creación de Clases (POO), el uso de Composer para gestionar dependencias y la separación de la lógica de negocio de la presentación mediante un motor de plantillas.

Durante el desarrollo he tenido que crear la estructura completa del proyecto, instalar y justificar las librerías necesarias, diseñar las clases principales y preparar las páginas que permiten tanto la creación de jugadores (uno por uno o en un paquete de datos “fake”), como la visualización de los mismos en una tabla.

También he añadido mejoras opcionales para que la aplicación sea más cómoda de usar, como una barra de navegación incluida en el layout general y algunos estilos adicionales en CSS.

El resultado final es una aplicación, que permite crear jugadores con sus datos básicos, generar códigos de barras únicos en formato EAN-13 y mostrar un listado con todos los jugadores registrados.

Además, he tomado y documentado las decisiones técnicas que he tenido que adoptar para poder realizar la práctica, como el cambio de algunas dependencias respecto al enunciado, para que quede claro el porqué de los cambios.

Estructura del proyecto

Para empezar la práctica lo primero que hice fue organizar la estructura del proyecto siguiendo el enunciado. Creé las carpetas principales y dentro de cada una fui colocando los ficheros que correspondían. La idea era tener todo bien separado con arquitectura limpia y también para cumplir con la parte de separar la lógica de negocio de la presentación.

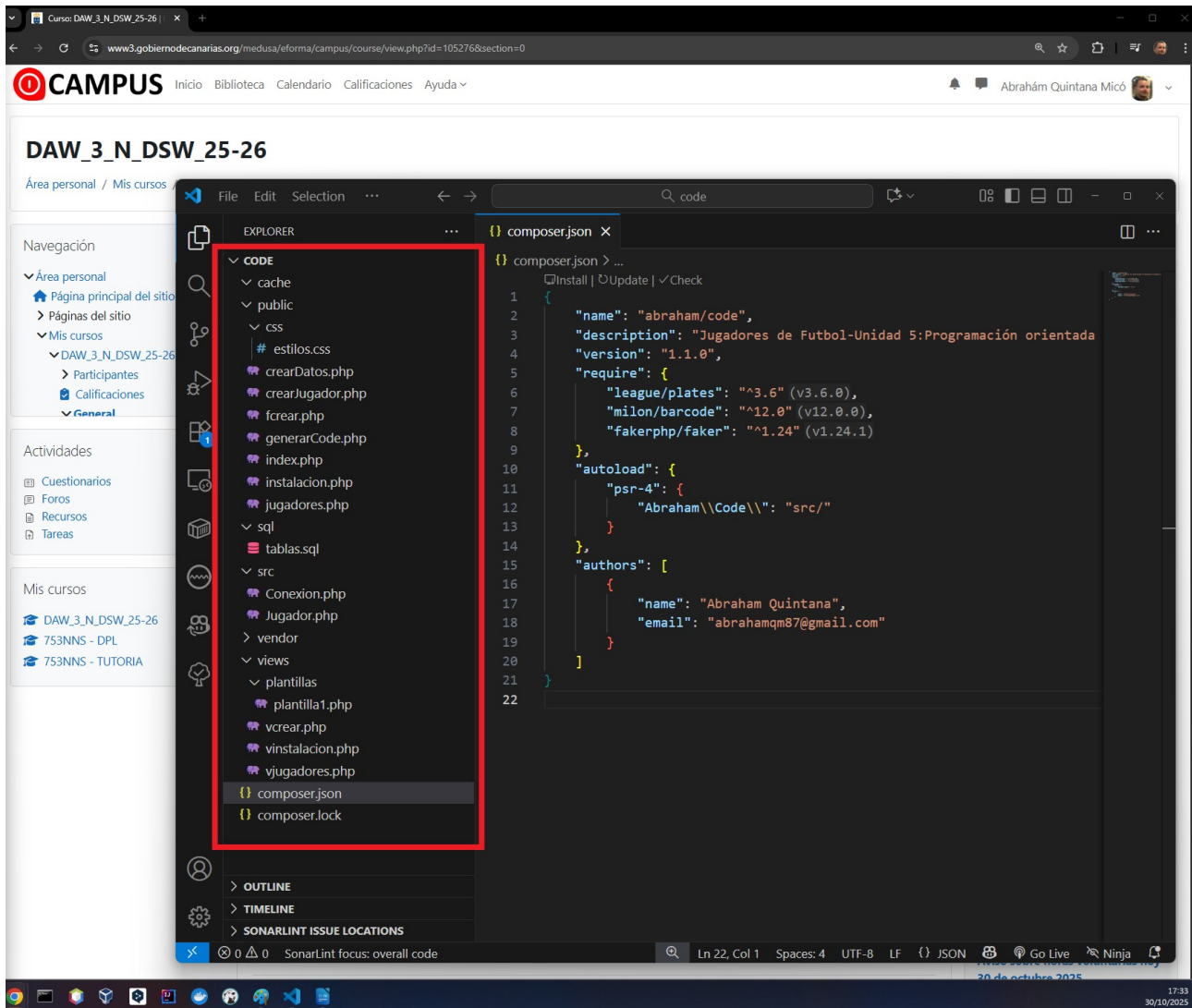
La estructura final quedó así:

- **cache/** → carpeta necesaria para que funcionen tanto el motor de plantillas como la librería de códigos de barras. En Linux hay que darle permisos 777.
- **public/** → aquí puse todas las páginas que se ejecutan desde el navegador: index.php, jugadores.php, fcrear.php, crearJugador.php, crearDatos.php, instalacion.php y generarCode.php.
- **public/css/** → carpeta donde guardé el archivo estilos.css, que contiene los estilos personalizados que fui añadiendo para mejorar el diseño de la aplicación.
- **sql/** → contiene el archivo con las sentencias SQL para crear la base de datos, la tabla jugadores y dar permisos al usuario.
- **src/** → aquí guardé las clases PHP. En mi caso Conexion.php para gestionar la conexión a la base de datos y Jugador.php para trabajar con la tabla de jugadores.
- **vendor/** → carpeta generada automáticamente por Composer al instalar las dependencias.
- **views/** → contiene las vistas que se cargan desde las páginas PHP.
 - Dentro de views hay una subcarpeta llamada **plantillas/** donde está la plantilla común plantilla1.php.
 - También están las vistas vcrear.php, vjugadores.php y vinstalacion.php, que se usan en cada una de las páginas principales.

Además, en la raíz del proyecto está el archivo composer.json con la configuración de las dependencias, y el composer.lock que se genera automáticamente para fijar las versiones.

Con esta estructura ya tenía la base sobre la que ir construyendo el resto de la aplicación.

Estructura final:



Dependencias instaladas y justificación de cambios

Para poder desarrollar la práctica tuve que instalar varias dependencias con Composer.

El enunciado pedía unas concretas, pero en algunos casos he tenido que hacer cambios porque las versiones originales daban problemas de compatibilidad con mi entorno. A continuación explico cada una y el motivo de la elección:

- **Motor de plantillas:** en el enunciado se pedía `philo/laravel-blade`, pero al probarlo me encontré con errores y dependencias necesarias que no podía usar con la versión de PHP que estoy utilizando 8.4, por ese motivo decidí sustituirlo por `league/plates`, que es un motor de plantillas fácil de integrar y sobre todo, no tenía problemas para instalarla. Con Plates he podido mantener la separación entre la lógica y la presentación, que era lo importante, y además me ha permitido trabajar con una plantilla común para todas las vistas.
- **Generador de códigos de barras:** aquí sí utilicé lo que pedía el enunciado, `milon/barcode`. Esta librería me permitió generar códigos de barras en formato EAN-13, que es el que se pedía. Pude mostrar los códigos directamente en la vista de jugadores sin problema.
- **Generador de datos de prueba:** el enunciado pedía `fzaninotto/faker`, pero esa librería ya no se mantiene. En su lugar instalé `fakerphp/faker`. Funciona igual que la original, así que pude generar nombres, apellidos, teléfonos y demás datos de ejemplo sin problemas.
- **Autoload optimizado:** Configuré el autoload de Composer para que las clases de `src/` se carguen automáticamente. Esto me ahorró tener que hacer `require` manuales y mantiene el proyecto más limpio.

En resumen, aunque cambié dos de las dependencias respecto al enunciado, lo hice por motivos de compatibilidad y mantenimiento. El resultado final es el mismo: tengo un motor de plantillas funcional, un generador de códigos de barras y un generador de datos de prueba.

En el pantallazo del punto anterior se puede ver el fichero `composer.json` con las dependencias instaladas

Explicación de los ficheros desarrollados

En este apartado voy a explicar de manera sencilla qué hace cada uno de los ficheros que forman parte del proyecto. La idea es que quede claro el papel de cada archivo dentro de la aplicación.

- **public/index.php**: es la página de inicio. Aquí compruebo si la tabla de jugadores tiene datos o no. Si está vacía redirige a instalacion.php para poder crear datos de ejemplo, y si ya hay jugadores registrados carga directamente jugadores.php.
- **public/jugadores.php**: muestra el listado de jugadores en una tabla. Llama a la vista vjugadores.php y le pasa los datos obtenidos desde la clase Jugador. Desde aquí también se puede acceder al formulario para crear un nuevo jugador.
- **public/fcrear.php**: es el formulario para crear un jugador. Carga la vista vcrear.php, donde se muestran los campos de nombre, apellidos, nacionalidad, teléfono, fecha de nacimiento, dorsal, posición y el código de barras. El campo del código de barras está en solo lectura y se puede generar con un botón que enlaza a generarCode.php.

Ojo, si hemos rellenado datos en el formulario, al pulsar el botón se recarga la página y se pierden los datos introducidos hasta el momento. Por eso puse un aviso en la parte superior del formulario. Podría haberlo solucionado persistiendo los datos hasta el momento en una cookie o en la sesión, pero no formaba parte de la práctica y, ya de por sí, es bastante larga y compleja en cuanto al tiempo de trabajo que requiere.

- **public/crearJugador.php**: procesa los datos enviados desde el formulario. Aquí valido que los campos obligatorios no estén vacíos y que el dorsal no se repita. Si todo está correcto, creo un objeto de la clase Jugador y llamo a su método insertar para guardar el nuevo registro en la base de datos.
- **public/crearDatos.php**: genera datos de ejemplo utilizando la librería Faker. Esto me sirvió para comprobar que la aplicación funcionaba correctamente antes de introducir datos reales y para probar esta dependencia que me parece bastante útil. Se crean 5 jugadores con datos aleatorios y se insertan en la base de datos.
- **public/instalacion.php**: muestra un botón que permite insertar los datos de ejemplo que se generan en public/crearDatos.php. Llama a la vista vinstalacion.php y esta a su vez a la creación de los datos.
- **public/generarCode.php**: genera un código de barras válido en formato EAN-13 que no exista en la base de datos. Este código se devuelve al formulario de creación para que el campo de código de barras se complete automáticamente.

- **public/css/estilos.css**: contiene los estilos que he añadido para mejorar la presentación. Aquí definí el ancho del contenido principal, los estilos de la tabla y la barra de navegación.
- **sql/tablas.sql**: archivo con las sentencias SQL necesarias para crear la base de datos, la tabla de jugadores y dar permisos al usuario.
- **src/Conexion.php**: clase que se encarga de abrir la conexión con la base de datos utilizando PDO. La hice estática para poder llamarla fácilmente desde cualquier parte del proyecto. Además es un singleton que no va a crear más de una conexión si ya existe una.
- **src/Jugador.php**: clase principal que representa a un jugador. Aquí están los métodos constructor, para insertar jugadores y obtener la lista de jugadores. También añadí la lógica para que si la fecha de nacimiento o el dorsal están vacíos se guarden como NULL.
- **views/plantillas/plantilla1.php**: es la plantilla común que utilizan todas las vistas. Aquí puse la cabecera HTML, la barra de navegación y la estructura básica de la página.
- **views/vcrear.php**: vista que contiene el formulario para crear un jugador. Se apoya en la plantilla común para mantener el mismo diseño.
- **views/vinstalacion.php**: vista sencilla que muestra el botón para instalar los datos de ejemplo.
- **views/vjugadores.php**: vista que muestra la tabla con el listado de jugadores. Si un jugador no tiene dorsal aparece el texto "Sin asignar".

Con todos estos ficheros la aplicación queda organizada en capas: la lógica de negocio está en las clases de src, la presentación en las vistas de views, y las páginas de public sirven de punto de entrada para el navegador.

Además para controlar los errores incluí try-catch, que me ayudo durante el desarrollo a solucionar los problemas que tuve.

Explicación de cada pantalla

En este apartado incluyo las capturas que pide el enunciado y explico brevemente qué se ve en cada una de ellas.

- **Estructura del proyecto:** aquí se muestra la organización de todas las carpetas y archivos que forman la aplicación. Se puede ver claramente la separación entre public, src, views, sql, vendor, cache y la carpeta de estilos. (Ver el pantallazo de Estructura del proyecto).
- **Formulario para crear jugador (fcrear.php):** en esta pantalla aparece el formulario con todos los campos necesarios para dar de alta un jugador. Incluye nombre, apellidos, nacionalidad, teléfono, fecha de nacimiento, dorsal, posición y el código de barras. El código de barras se genera automáticamente con un botón que llama a generarCode.php. Tiene campos obligatorios y al crear el jugador redirige a la página jugadores donde se ve el resultado. Pantalla con datos rellenos para crear el primero:

The screenshot displays a web browser window with two tabs. The background tab is 'Curso: DAW_3_N_DSW_25-26' from 'www3.gobiernodecanarias.org/medusa/eforma/campus/course/view.php?id=105276§ion=0'. The foreground tab is 'Crear Jugador' from 'localhost/ut5/code/public/fcrear.php?barcode=9100082084323'. The application has a dark blue header with the 'CAMPUS' logo and navigation links: Inicio, Biblioteca, Calendario, Calificaciones, Ayuda. A user profile for 'Abrahám Quintana Micó' is in the top right. A left sidebar contains a 'Navegación' menu with 'Área personal' (containing 'Página principal del sitio', 'Páginas del sitio', 'Mis cursos' with sub-items 'DAW_3_N_DSW_25-26', 'Participantes', 'Calificaciones', and 'General'), 'Actividades' (Cuestionarios, Foros, Recursos, Tareas), and 'Mis cursos' (DAW_3_N_DSW_25-26, 753NNS - DPL, 753NNS - TUTORIA). The main content area has a blue header 'Aplicación de Jugadores' with buttons 'Listado', 'Crear Jugador', and 'Instalación'. Below this is a white box titled 'Crear Jugador' with a warning: 'Ojo: No olvides crear el código de barras pulsando el boton antes de rellenar el formulario o se perderán los datos al recargarse la página!'. The form contains fields for 'Nombre' (Abraham), 'Apellidos' (Quintana), 'Teléfono' (666777888), 'Nacionalidad' (español), 'Fecha de nacimiento' (30/10/2025), 'Dorsal' (33), and 'Posición' (Portero). A 'Generar Barcode' button is next to a field containing '9100082084323'. At the bottom are 'Crear', 'Limpiar', and 'Volver' buttons. The system clock at the bottom right shows '30 de octubre 2025' and '18:36'.

Abraham Quintana Micó

- **Listado de jugadores (jugadores.php)**: esta pantalla muestra la tabla con todos los jugadores que hay en la base de datos. Se ven los datos básicos y también el código de barras en formato EAN-13. Permite acceder a la creación de un jugador según se pide en el ejercicio. Además, si un jugador no tiene dorsal aparece el texto “Sin asignar”. He añadido un 2º jugador sin dorsal para que se vea eso.

The screenshot shows a web application titled "Aplicación de Jugadores" running on a local server. The application has a sidebar with navigation links and a main content area displaying a list of players.

Navigation Sidebar:

- Inicio
- Biblioteca
- Calendario
- Calificaciones
- Ayuda
- Área personal
- Página principal del sitio
- Páginas del sitio
- Mis cursos
- DAW_3_N_DSW_25-26
- Participantes
- Calificaciones
- General
- Actividades
- Cuestionarios
- Foros
- Recursos
- Tareas
- Mis cursos
- DAW_3_N_DSW_25-26
- 753NNS - DPL
- 753NNS - TUTORIA

Main Content Area:

Aplicación de Jugadores

Buttons: Listado, Crear Jugador, Instalación

Listado de Jugadores

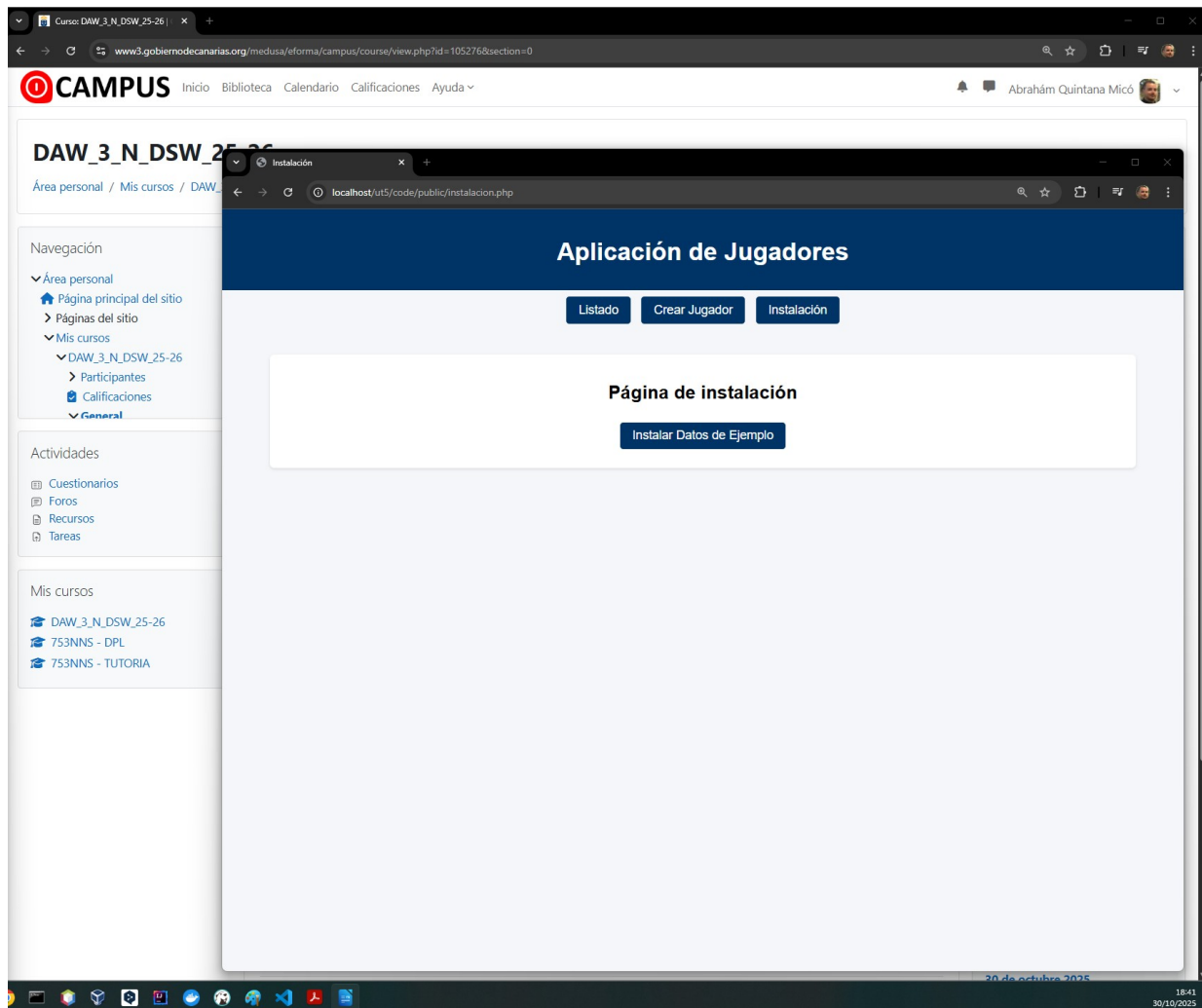
+ Nuevo Jugador

Nombre Completo	Teléfono	Nacionalidad	Fecha Nacimiento	Posición	Dorsal	Código de Barras
Abraham Quintana	666777888	español	2025-10-30	Portero	33	9100082084323
pepe benabente	666999888	chino	2025-10-01	Defensa	Sin asignar	7983320072410

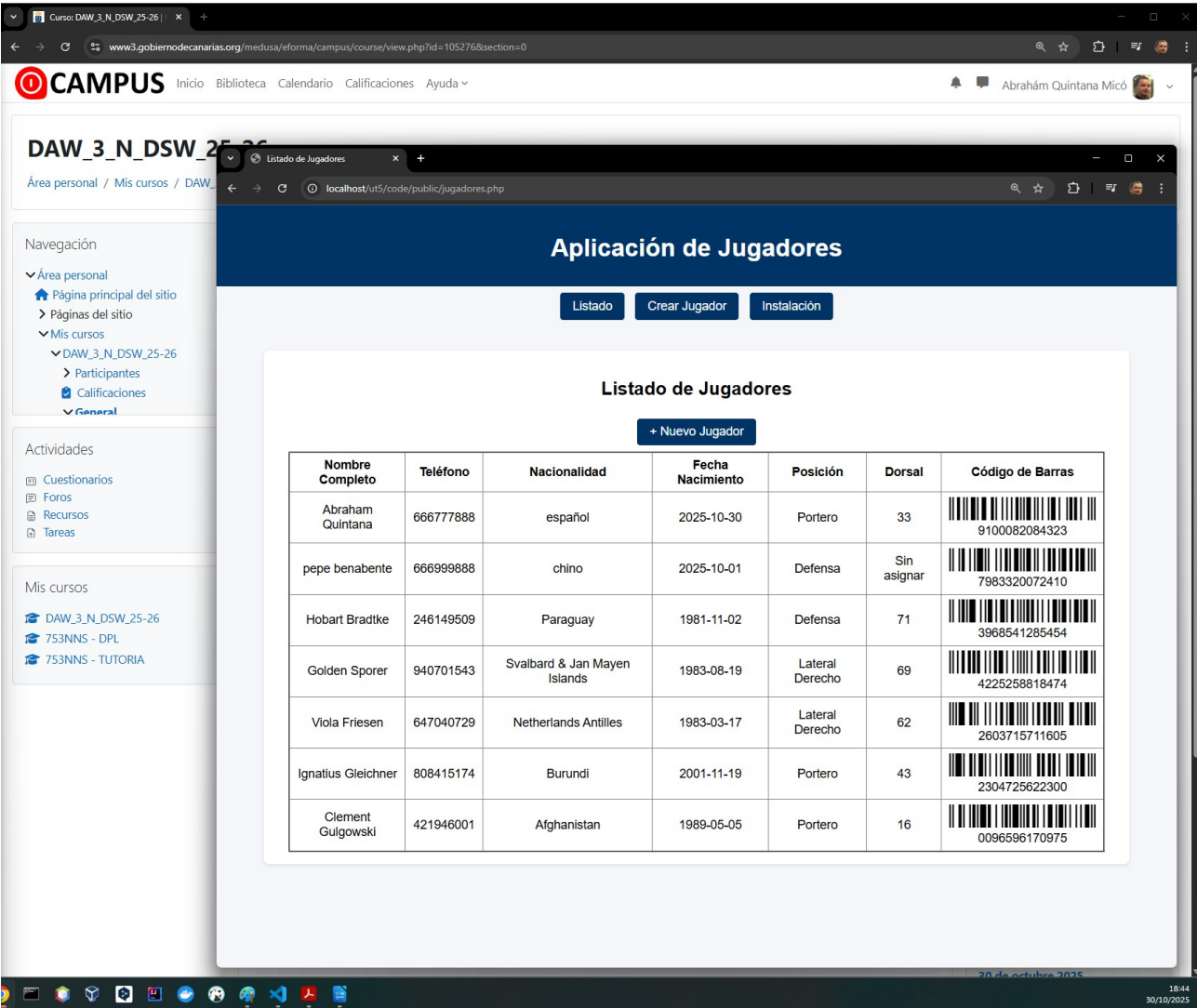
- **Pantalla de crearDatos.php:** se accede si vamos al index y no hay jugadores o si pulsamos en el nav donde pone instalación. Aquí se puede comprobar el uso de la librería Faker. Al pulsar el botón se generan cinco jugadores con datos aleatorios (nombres, apellidos, teléfonos, etc.) y se insertan en la base de datos.

Al pulsar el botón, llama a crearDatos y se insertan los jugadores, luego redirige a la página jugadores para ver el resultado automáticamente.

Página de instalación:



Esta captura muestra el resultado en la página jugadores, demostrando que la dependencia funciona correctamente con los 5 nuevos jugadores:



Mejoras añadidas y justificación

Además de cumplir con los requisitos básicos del enunciado, he añadido algunas mejoras que considero necesarias y, que hacen la aplicación más completa y agradable de usar.

- **Barra de navegación común:** añadí una barra de navegación en la plantilla principal para que todas las páginas tengan acceso rápido a las secciones más importantes. Esto mejora la usabilidad y da coherencia al diseño, ya que el usuario siempre sabe cómo ir al listado, al formulario de creación o a la página de instalación para crear datos ficticios.
- **Estilos CSS personalizados:** creé un archivo estilos.css donde definí algunos estilos básicos para que la aplicación no se viera desagradable. Ajusté el ancho del contenido principal, mejoré la presentación de la tabla de jugadores, ... en general toda la aplicación.
- **Control de valores nulos:** en la clase Jugador añadí la lógica para que si la fecha de nacimiento o el dorsal están vacíos se guarden como NULL en la base de datos. Esto evita errores y cumple con lo que pedía el enunciado, ya que esos campos no son obligatorios.
- **Uso de try-catch:** durante el desarrollo incluí bloques try-catch en los puntos clave de la aplicación. Esto me ayudó a detectar errores de conexión o de inserción y a entender mejor qué estaba fallando en cada momento.

Con estas mejoras la aplicación no solo cumple con lo que pedía el enunciado, sino que también resulta más clara, más usable y más robusta frente a posibles errores.

Conclusión

En mi opinión esta práctica me ha servido sobre todo para aprender a usar algunas dependencias y a manejar el gestor de paquetes Composer. Sin embargo, creo que no se centra lo suficiente en la programación orientada a objetos. Apenas se utilizan dos clases y no se trabajan conceptos como interfaces, herencia o la sobreescritura de métodos, que son la base de la POO.

De nuevo me encontré con una práctica que más bien parece un proyecto de final de asignatura que una tarea para hacer en una semana. Ha sido muy larga y exigente en cuanto al tiempo de dedicación. En mi caso calculo que he tenido que invertir más de 35 horas para poder completarla, sin contar la memoria y, de ese tiempo unas 5 horas fueron solo para intentar resolver los problemas con las librerías que al final tuve que superar buscando alternativas.

Bajo mi experiencia como programador, creo que hubiera sido más útil una práctica más clásica de POO, por ejemplo crear una clase básica como Vehículo que implemente una interfaz, y a partir de ella extender varias clases hijas (coche, moto, camión, etc.), sobreescribir métodos y trabajar con atributos encapsulados y modificadores de acceso (public, private, protected). Ese tipo de ejercicios muestran de forma más clara cómo la POO permite reaprovechar código, encapsular lógica y estructurar mejor los proyectos.

Aun así, supongo que el objetivo era enfrentarnos a un proyecto más completo y a la integración de dependencias externas. Mi intención con este comentario no es criticar, sino aportar mi punto de vista como alumno y programador, con la esperanza de que pueda servir de ayuda para mejorar la experiencia de otros estudiantes en el futuro. Confío en que esta opinión personal no afecte a la nota final de la práctica.

Bibliografía

- **league/plates**: <https://packagist.org/packages/league/plates>
- **fakerphp/faker**: <https://fakerphp.org/>
- **milon/barcode**: <https://github.com/milon/barcode>

También revisé foros y páginas de soporte relacionadas con problemas de compatibilidad de dependencias en Composer y con la instalación en entornos con PHP 8.4, lo que me ayudó a entender por qué algunas librerías no podían usarse y a justificar los cambios que realicé.